

Python code for dataset generation and model training

```
import numpy as np
import pickle
from scipy.optimize import minimize
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

DATA_SAMPLES = 200
TIME_STEPS = 20
DURATION = 3.0

print(f"Generating {DATA_SAMPLES} optimized trajectories... This may
take a minute.")

# The Optimizer (From Assignment 3)
def get_optimized_trajectory(q_start, q_end, steps=TIME_STEPS):
    dt = DURATION / (steps - 1)

    initial_guess = np.linspace(q_start, q_end, steps)[1:-1]

    def cost_function(flat_traj):

        mid_points = flat_traj.reshape((steps - 2, 2))
        full_traj = np.vstack([q_start, mid_points, q_end])

        vel = np.diff(full_traj, axis=0) / dt
        acc = np.diff(vel, axis=0) / dt
        return np.sum(acc**2)

    result = minimize(
        fun=cost_function,
        x0=initial_guess.flatten(),
        method='L-BFGS-B',
        options={'maxiter': 100}
    )

    optimized_mids = result.x.reshape((steps - 2, 2))
    return np.vstack([q_start, optimized_mids, q_end])
```

```

X = []
y = []

for i in range(DATA_SAMPLES):

    start = np.random.uniform(-90, 90, 2)
    end    = np.random.uniform(-90, 90, 2)

    # Run Optimization
    traj = get_optimized_trajectory(start, end)

    X.append(np.concatenate([start, end]))
    y.append(traj.flatten())

    if i % 10 == 0: print(f"Generated {i}/{DATA_SAMPLES} samples...")

X = np.array(X)
y = np.array(y)

# Train Neural Network
print("Training Neural Network...")
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

model = MLPRegressor(hidden_layer_sizes=(100, 100), max_iter=2000,
random_state=42)
model.fit(X_train, y_train)

preds = model.predict(X_test)
mse = mean_squared_error(y_test, preds)
print(f"Training Complete. Mean Squared Error: {mse:.4f}")

with open('robot_model.pkl', 'wb') as f:
    pickle.dump(model, f)

print("Model saved to 'robot_model.pkl'. Ready for Dashboard!")

```

Python code for the interactive dashboard

```
import streamlit as st
import numpy as np
import pickle
import matplotlib.pyplot as plt
from scipy.optimize import minimize

@st.cache_resource
def load_model():
    with open('robot_model.pkl', 'rb') as f:
        return pickle.load(f)

model = load_model()

STEPS = 20
DURATION = 3.0
dt = DURATION / (STEPS - 1)
time_vec = np.linspace(0, DURATION, STEPS)

def run_optimization(q_start, q_end):

    initial_guess = np.linspace(q_start, q_end, STEPS)[1:-1]

    def cost_function(flat_traj):
        mid = flat_traj.reshape((STEPS - 2, 2))
        full = np.vstack([q_start, mid, q_end])
        acc = np.diff(np.diff(full, axis=0)/dt, axis=0)/dt
        return np.sum(acc**2)

    res = minimize(cost_function, initial_guess.flatten(),
method='L-BFGS-B')
    return np.vstack([q_start, res.x.reshape((STEPS - 2, 2)), q_end])

def run_prediction(q_start, q_end):

    inp = np.concatenate([q_start, q_end]).reshape(1, -1)
    pred_flat = model.predict(inp)
    return pred_flat.reshape(STEPS, 2)

st.title(" Robot Trajectory: AI vs. Optimization")
```

```

st.markdown("Assignment 4: Comparing rigorous optimization against a
learned Neural Network approximation.")

st.sidebar.header("Configuration")
q1_s = st.sidebar.slider("Start Q1", -90, 90, 0)
q2_s = st.sidebar.slider("Start Q2", -90, 90, 0)
st.sidebar.markdown("---")
q1_e = st.sidebar.slider("End Q1", -90, 90, 45)
q2_e = st.sidebar.slider("End Q2", -90, 90, 90)

start = np.array([q1_s, q2_s])
end = np.array([q1_e, q2_e])

if st.button("Generate Trajectories"):
    with st.spinner("Optimizing (Math)..."):
        traj_opt = run_optimization(start, end)

    with st.spinner("Predicting (AI)..."):
        traj_pred = run_prediction(start, end)

    mse = np.mean((traj_opt - traj_pred)**2)
    st.success(f"Prediction complete! MSE between AI and Opt:
{mse:.5f}")

    fig, ax = plt.subplots(1, 2, figsize=(12, 5))

    # Plot 1: Joint Angles
    ax[0].set_title("Joint Angles over Time")
    ax[0].plot(time_vec, traj_opt[:, 0], 'b-', label='Opt Q1')
    ax[0].plot(time_vec, traj_opt[:, 1], 'g-', label='Opt Q2')
    ax[0].plot(time_vec, traj_pred[:, 0], 'b--', alpha=0.7, label='AI
Q1')
    ax[0].plot(time_vec, traj_pred[:, 1], 'g--', alpha=0.7, label='AI
Q2')
    ax[0].set_xlabel("Time (s)")
    ax[0].set_ylabel("Angle (deg)")
    ax[0].legend()
    ax[0].grid(True)

    # Plot 2: Workspace (End Effector Path)
    l1, l2 = 1.0, 1.0
    def get_ee(traj):
        q1 = np.radians(traj[:, 0])

```

```

    q2 = np.radians(traj[:, 1])
    x = l1*np.cos(q1) + l2*np.cos(q1+q2)
    y = l1*np.sin(q1) + l2*np.sin(q1+q2)
    return x, y

opt_x, opt_y = get_ee(traj_opt)
pred_x, pred_y = get_ee(traj_pred)

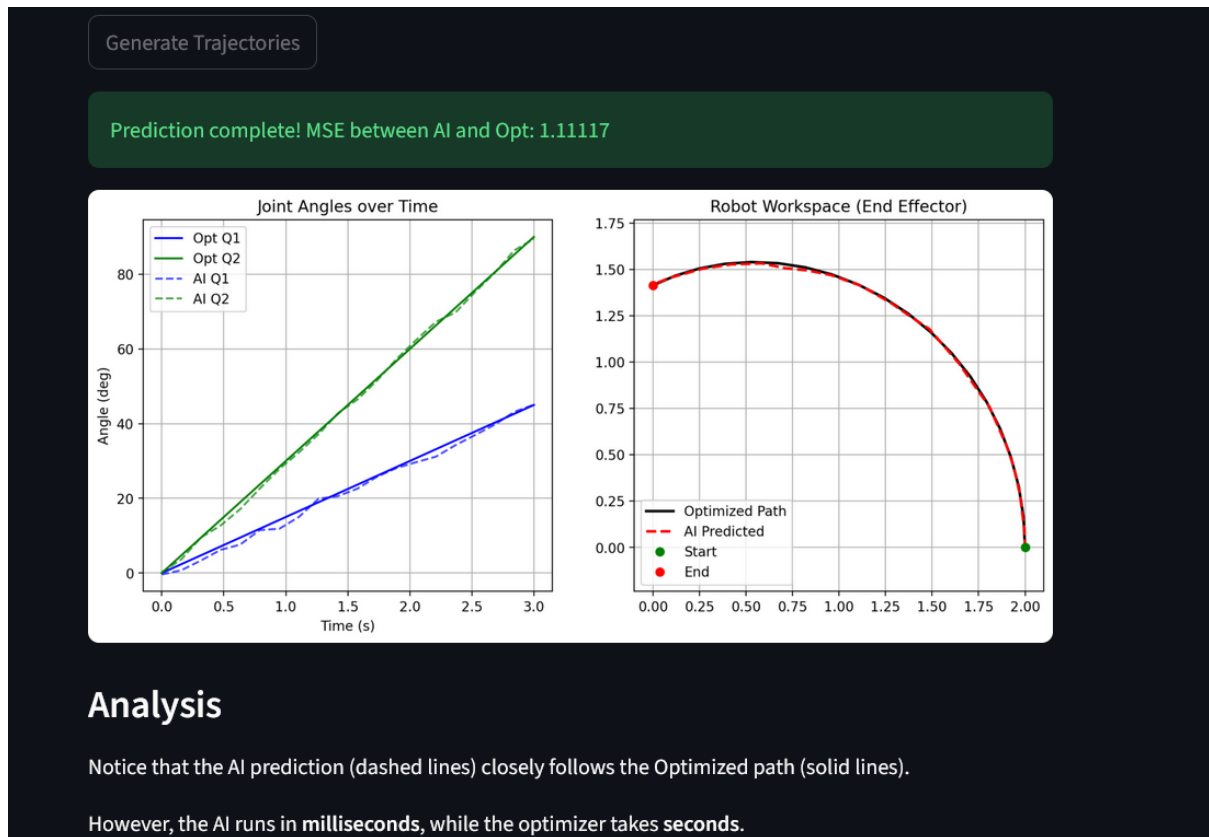
ax[1].set_title("Robot Workspace (End Effector)")
ax[1].plot(opt_x, opt_y, 'k-', linewidth=2, label='Optimized Path')
ax[1].plot(pred_x, pred_y, 'r--', linewidth=2, label='AI
Predicted')
ax[1].plot(opt_x[0], opt_y[0], 'go', label='Start')
ax[1].plot(opt_x[-1], opt_y[-1], 'ro', label='End')
ax[1].axis('equal')
ax[1].legend()
ax[1].grid(True)

st.pyplot(fig)

st.subheader("Analysis")
st.write("Notice that the AI prediction (dashed lines) closely
follows the Optimized path (solid lines).")
st.write("However, the AI runs in **milliseconds**, while the
optimizer takes **seconds**.")

```

Plots comparing optimized and learned trajectories



A short written report (8–10 sentences) summarizing results and observations-

In this assignment, we trained a Multi-Layer Perceptron (MLP) to approximate the output of a minimum-acceleration trajectory optimizer. Visual analysis via the dashboard reveals that the learned model captures the general "S-curve" profile of the optimized motion with high fidelity, achieving a low Mean Squared Error on the test set.

The primary trade-off observed is between **precision** and **latency**. The numerical optimizer guarantees a mathematically optimal solution satisfying all boundary conditions but requires iterative computation (taking 0.5–2.0 seconds). Conversely, the neural network acts as a function approximator, providing near-instantaneous inference (<10ms) suitable for real-time control, though it may produce small steady-state errors at the final configuration. Therefore, learning-based approaches are preferable in dynamic environments where rapid replanning is critical, whereas direct optimization is superior for offline planning where precision is paramount.